Parameter Tuning of Evolutionary Algorithms: Generalist vs. Specialist

S.K. Smit and A.E. Eiben

Vrije Universiteit Amsterdam
The Netherlands
{sksmit,gusz}@cs.vu.nl
http://mobat.sourceforge.net

Abstract. Finding appropriate parameter values for Evolutionary Algorithms (EAs) is one of the persistent challenges of Evolutionary Computing. In recent publications we showed how the REVAC (Relevance Estimation and VAlue Calibration) method is capable to find good EA parameter values for single problems. Here we demonstrate that REVAC can also tune an EA to a set of problems (a whole test suite). Hereby we obtain robust, rather than problem-tailored, parameter values and an EA that is a 'generalist, rather than a 'specialist. The optimized parameter values prove to be different from problem to problem and also different from the values of the generalist. Furthermore, we compare the robust parameter values optimized by REVAC with the supposedly robust conventional values and see great differences. This suggests that traditional settings might be far from optimal, even if they are meant to be robust.

Keywords: parameter tuning, algorithm design, test suites, robustness.

1 Background and Objectives

Finding appropriate parameter values for evolutionary algorithms (EA) is one of the persisting grand challenges of the evolutionary computing (EC) field. As explained by Eiben *et al.* in [8] this challenge can be addressed before the run of the given EA (parameter tuning) or during the run (parameter control). In this paper we focus on parameter tuning, that is, we are seeking good parameter values off-line and use these values for the whole EA run. In today's practice, this tuning problem is usually 'solved' by conventions (mutation rate should be low), ad hoc choices (why not use uniform crossover), and experimental comparisons on a limited scale (testing combinations of three different crossover rates and three different mutation rates). Until recently, there were not many workable alternatives. However, by the developments over last couple of years now there are a number of tuning methods and corresponding software packages that enable EA practitioners to perform tuning without much effort. In particular, REVAC [10,13] and SPOT [3,5,4] are well developed and documented.

The main objective of this paper is to illustrate the advantage of using tuning algorithms in terms of improved EA performance. To this end, we will select

C. Di Chio et al. (Eds.): EvoApplications 2010, Part I, LNCS 6024, pp. 542–551, 2010.
© Springer-Verlag Berlin Heidelberg 2010

a set of test functions and compare a benchmark EA (with robust parameter values set by 'common wisdom') with an EA whose parameters are tuned for this set of functions. A second objective is to compare specialist EAs (that are tuned on one of our test functions) with a generalist EA (that is tuned on the whole set of test functions). For this comparison we will look at the performance of the EAs as well as the tuned parameter values. Furthermore, we want to show what kind of problems rise when tuning evolutionary algorithms. At the end we hope to provide a convincing showcase justifying the use of a tuning algorithm and to obtain novel insights regarding the good parameter values.

2 Parameters, Tuners, and Utility Landscapes

To obtain a detailed view on parameter tuning we distinguish three layers: the application layer, the algorithm layer, and the design or tuning layer. The lower part of this three-tier hierarchy consists of a problem on the application layer (e.g., the traveling salesman problem) and an EA (e.g., a genetic algorithm) on the algorithm layer trying to find an optimal solution for this problem. Simply put, the EA is iteratively generating candidate solutions (e.g., permutations of city names) seeking one with maximal fitness. The upper part of the hierarchy contains a tuning method that is trying to find optimal parameter values for the EA on the algorithm layer. Similarly to the lower part, the tuning method is iteratively generating parameter vectors seeking one with maximal quality, where the quality of a given parameter vector is based on the performance of the EA using the values of it. To avoid confusion we use the term *utility*, rather than fitness, to denote the quality of parameter vectors. Table 1 provides a quick overview of the related vocabulary.

Using this nomenclature we can define the *utility landscape* as an abstract landscape where the locations are the parameter vectors of an EA and the height reflects utility, based on any appropriate notion of EA performance. It is obvious that fitness landscapes –commonly used in EC– have a lot in common with utility landscapes as introduced here. To be specific, in both cases we have a search space (candidate solutions vs. parameter vectors), a quality measure (fitness vs. utility) that is conceptualized as 'height', and a method to assess the quality of a point in the search space (evaluation vs. testing). Finally, we have a search method (an evolutionary algorithm vs. a tuning procedure) that is seeking for a point with maximum height.

Table 1.

	problem solving	parameter tuning
Method at work	evolutionary algorithm	tuning procedure
Search space	solution vectors	parameter vectors
Quality	fitness	utility
Assessment	evaluation	testing